

实验1：RISC-V 引导与裸机启动

使用说明：本实验使用 `whu-oslab-skeleton/` 框架，你的所有代码都写在该框架中对应文件的 `TODO` 标注位置。不要修改 `kernel/include/` 中的头文件。

实验目标

本实验旨在打破“C 语言程序总是从 `main` 函数开始”的思维定势。在没有操作系统的“裸机（Bare-metal）”环境下，你将抛弃所有标准库，亲手编写底层引导代码，为 C 语言的运行创造基础条件，并最终利用物理内存地址直接操控硬件，在屏幕上输出 `"Hello OS from RISC-V Bare-metal!"`。

理论回顾：真机启动 vs QEMU 启动

在理论课中，一台物理计算机的启动（Boot）通常包含三个漫长的阶段：

1. **硬件自检 (POST)**: 主板通电，检查 CPU、内存、外设状态是否正常。
2. **引导加载 (Bootloader)**: 运行 BIOS/UEFI 或 OpenSBI 等固件，它们从硬盘主引导记录中读取 Bootloader（如 GRUB）。
3. **内核加载**: Bootloader 驱动硬盘，将操作系统内核 (Kernel) 从硬盘复制到内存，最后跳转到内核入口。

为什么我们在实验中感觉不到前面的步骤？

因为在这个实验中，**QEMU 模拟器充当了“物理魔法”的角色**。在启动 QEMU 时的参数包含 `-bios none -kernel kernel.elf`：

- `-bios none`：我们告诉 QEMU 不加载任何底层基础固件（如 OpenSBI），我们要让 CPU 直接处于最原始、特权级最高的机器模式（Machine Mode）。
- `-kernel kernel.elf`：QEMU 绕过了极其复杂的“驱动硬盘读文件”的过程，直接开启上帝视角，把我们编译好的内核可执行文件“凭空”铺到了虚拟物理内存的 `0x80000000` 地址。

理论与实践的鸿沟：启动的三大难题

在开始写代码前，我们要明白：当计算机刚通电点亮时，面临的是一片混沌的世界。要在这样的环境下跑起 C 语言，你必须亲自越过三道鸿沟：

1. **CPU 的迷茫**: 通电后，CPU 怎么知道去内存的哪个绝对位置找第一条指令？
2. **C 语言无法运行**: C 语言的局部变量和函数调用机制必须要有一块内存作为“**栈 (Stack)**”，而刚开机时，硬件根本没有“栈”的概念。
3. **没有屏幕输出**: 不能使用 `#include <stdio.h>` 和 `printf`。我们要怎么让屏幕显示字符？

本实验的任务线，就是为了逐一击破这三个难题。

★ 认识你的工具链

在做任何事之前，先确认工具链已安装可用：

```
1 riscv64-unknown-elf-gcc --version # 应输出版本信息
2 qemu-system-riscv64 --version    # 应输出 QEMU 版本
```

编译参数速查（后续 Makefile 会用到这些参数，这里帮你提前理解）：

参数	含义
<code>-nostdlib</code>	不链接任何 C 标准库（记住：裸机世界里没有 <code>libc</code> ！）
<code>-fno-builtin</code>	禁用编译器内置函数（如 <code>memcpy</code> ），避免它偷偷链接库函数
<code>-mcmodel=medany</code>	代码模型：支持访问任意64位地址范围（内核必须用这个）
<code>-march=rv64gc</code>	目标CPU：64位RISC-V，包含整数/乘除/原子/压缩指令集
<code>-mabi=lp64d</code>	ABI： <code>long</code> 和指针是64位，浮点使用硬件寄存器
<code>-T kernel.ld</code>	使用自定义链接脚本（而不是系统默认的）

★ 感知框架结构

打开 `whu-oslab-skeleton/`，本实验你只需要关注以下4个文件（其他文件是后续实验用的，现在忽略）：

```
1 whu-oslab-skeleton/
2 |─ Makefile           ← 本实验要设置 SRCS，然后用 make run 验收
3 |─ kernel.ld         ← 任务1：填写正确的起始地址
4 |─ kernel/
5 |   |─ boot/
6 |   |   |─ entry.S    ← 任务2：填写栈初始化汇编代码
7 |   |   |─ main.c     ← 任务4：填写第一行 C 语言代码
8 |   |─ driver/
9 |       |─ uart.c     ← 任务3：填写串口输出函数
```

任务指南与实施步骤

任务1：控制内核在哪里安家（链接脚本）

应对难题 1：CPU去哪找第一条指令？

在 QEMU 模拟器的“virt”主板规定中，当计算机启动初始化后，它的程序计数器 (PC) 会固定跳到物理内存地址 `0x80000000` 处，开始无脑执行第一条指令。

📁 目标文件： `kernel.ld`

打开该文件，找到 TODO 注释：

```
1  /* TODO [Lab1-任务1] :  
2    *   在下方填写正确的内核加载起始地址。  
3    */  
4    . = 0xDEADBEEF;    /* <-- 改成正确的内核物理起始地址 */
```

任务：将 `0xDEADBEEF` 改为 QEMU virt 平台规定的物理起始地址。

💡 提示：答案已经在上面的理论说明里了——QEMU 会把内核铺到 `0x` 地址。

✅ **验证方法：**完成任务2后，可以运行以下命令验证第一条指令是否在正确位置：

```
1  riscv64-unknown-elf-objdump -d kernel.elf | head -20  
2  # 输出应该显示第一条指令的地址是 0x80000000
```

任务2：搭建 C 语言运行的舞台（启动汇编）

应对难题 2：C 语言没有栈怎么跑？

📁 目标文件： `kernel/boot/entry.S`

打开该文件，你会看到 4 个 TODO 步骤，结构如下：

```
1  _entry:  
2      # TODO [步骤1]: la sp, ???      → 把 stack0 的地址加载到 sp  
3      # TODO [步骤2]: li a0, ???      → 把 4096 加载到 a0  
4      # TODO [步骤3]: add sp, sp, ???→ sp = sp + a0 (指向栈顶)  
5      # TODO [步骤4]: call ???        → 跳转到 C 函数 start_main
```

★ 为什么栈是“向下生长”的？

```
1  内存地址增大方向 ↑  
2  ┌──────────┐ ← stack0 + 4096 (这里是栈顶！sp 应该指向这里)  
3  │ 可用栈空间 │ 每次 push (函数调用)，sp 往下走  
4  │            │  
5  │            │  
6  └──────────┘ ← stack0 (底部，标签地址)  
7  内存地址减小方向 ↓
```

RISC-V 的栈从高地址向低地址生长。所以 `sp` 必须指向 `stack0 + 4096`（最高地址处），这样才有足够的空间向下扩展。

逐步填写：

- **步骤1：** `la sp, stack0` — `la` (Load Address) 把 `stack0` 标签的地址赋给 `sp`
- **步骤2：** `li a0, 4096` — `li` (Load Immediate) 把立即数 4096 赋给 `a0`

- 步骤3: `add sp, sp, a0` — `sp = stack0 + 4096`, 现在 `sp` 指向栈顶
 - 步骤4: `call start_main` — 跳入 C 语言主函数
-

任务3: 使用内存地址操控硬件 (串口驱动)

应对难题 3: 没有 `printf` 怎么让屏幕显示内容?

📁 目标文件: `kernel/driver/uart.c`

★ MMIO 原理一分钟速成

硬件设计者在芯片制造时, 就规定: 向地址 `0x10000000` 写一个字节, 这个字节会自动发到串口电路, 打印到终端。这种机制叫做 **MMIO (内存映射I/O)**。

```
1 你的代码: *((volatile char*)0x10000000) = 'H';
2           ↓
3  CPU: 向物理地址 0x10000000 发出写请求
4           ↓
5  UART硬件: 收到信号, 把字符 'H' 发送到串口
6           ↓
7  终端: 显示 H
```

`volatile` 关键字的作用: 告诉编译器, 这个内存访问有真实的硬件副作用, 不能优化掉! 否则编译器可能认为“你反复写同一个地址, 只留最后一次就够了”, 把中间的写操作全删掉。

打开文件, 依次填写两个 TODO:

步骤1 (`uart_putc`) :

```
1 // TODO: 将字符 c 写入 UART0 数据寄存器 (偏移量 0)
2 // 把下面这行 (void)c; 删掉, 替换为:
3 *Reg(0) = c;
```

步骤2 (`uart_puts`) :

```
1 // TODO: 遍历字符串, 逐字符调用 uart_putc
2 // 把 (void)s; 删掉, 替换为:
3 while (*s) {
4     uart_putc(*s);
5     s++;
6 }
```

任务4: 拼凑主线并通电试运行

📁 目标文件: `kernel/boot/main.c`

```
1 // TODO [Lab1-任务4]:
2 // 实现 start_main() 函数:
3 // 1. 调用 uart_puts() 打印 "Hello OS from RISC-V Bare-metal!\n"
```

```
4 // 2. 然后进入 while(1) 死循环
```

填写后 `start_main()` 应该是：

```
1 void start_main() {
2     uart_puts("Hello OS from RISC-V Bare-metal!\n");
3     while(1);
4 }
```

任务5 — 配置构建系统并运行

 目标文件： `Makefile`

找到 `SRCS` 变量，确认它包含 Lab1 的三个源文件（该框架已配置好，确认即可）：

```
1 SRCS = \
2     kernel/boot/entry.S \
3     kernel/driver/uart.c \
4     kernel/boot/main.c
```

现在运行：

```
1 # 在 whu-oslab-skeleton 目录下
2 make          # 编译，生成 kernel.elf
3 make run      # 启动 QEMU，观察输出
```

预期输出：

```
1 Hello OS from RISC-V Bare-metal!
```

退出 QEMU：按 `Ctrl+A`，然后按 `X`。

避坑与故障排除策略 (Debugging)

如果屏幕一片死寂，不要慌，这是写操作系统最享受的时光。此时没有任何编译器报错，代码逻辑死在了黑盒子里，你需要像侦探一样排查：

分段定位法：

是否卡在了第一行汇编代码 `_entry` 处？

1. 用 GDB 调试看看程序的第一条指令是否真的落地在 `0x80000000` ！

```
1 # 终端1：启动 QEMU 并冻结等待调试
2 qemu-system-riscv64 -machine virt -bios none -kernel kernel.elf -nograph
3
4 # 终端2：启动 GDB 连接调试
5 riscv64-unknown-elf-gdb kernel.elf
6 (gdb) target remote :1234
```

```
7 (gdb) info registers pc      # 检查 PC 是否是 0x80000000
8 (gdb) si                    # 单步执行，观察是否有异常
```

C语言的栈是否分配成功？

1. 在 GDB 下单步进入汇编 (`si`)，在执行到 `call start_main` 之前，停下来用命令 `info registers sp` 检查一下当前栈指针。看看 `sp` 是不是等于 `stack0` 的地址加上了 `4096`？

是不是打印功能写错了？

2. 如果 GDB 单步跟踪发现成功切入进 `start_main` 里面去了，却没有在终端打印出任何东西。请去检查 `uart.c` 里的 `*Reg(0) = c` 中的 `UART0_BASE` 是不是 `0x10000000`。

进阶实验验收

实验完成后需能解答以下问题：

1. 请观察 `entry.S` 的代码，如果不给 `sp` 赋值，直接就 `call start_main`，QEMU 终端会发生什么样的运行崩溃或奇怪现象？请从原理解释为何会这样。
2. 我们在 C 语言直接写指针赋值 `*Reg(0) = c;` 就能精准控制主板外部硬件。在宏定义指针时，如果你漏加了 `volatile` 关键字，当你将来用 gcc 的 `-O3` 开启高级优化时，这段串口发送代码可能会被编译器怎么“自作聪明”地修改掉，导致什么严重的后果？

★ 本实验完成后的全貌

`main.c` 最终形态：

```
1 extern void uart_puts(char *s);
2
3 void start_main() {
4     uart_puts("Hello OS from RISC-V Bare-metal!\n");
5     while(1);
6 }
```

项目文件状态 (☒ = 已完成实现, ☐ = 后续实验完成)：

```
1 whu-oslab-skeleton/
2 └─ kernel.ld          ✓ (起始地址已填写)
3 └─ kernel/boot/
4   └─ entry.S          ✓ (栈初始化已完成)
5   └─ main.c            ✓ (print 调用已完成)
6   └─ start.c           ○ (Lab4 实现)
7 └─ kernel/driver/
8   └─ uart.c            ✓ (putc/puts 已完成)
9   └─ console.c         ○ (Lab2 实现)
10 └─ kernel/mm/...      ○ (Lab3 实现)
```